

Cours

Capacités numériques

Filière CPGE

proposé sur Tout pour CPGérer

rédigé par A. Cornet

Version de octobre 2026

Table des matières

1 Résolution d'équations **4**
- I Dichotomie . . . 4
  - I.1 Principe . . . 4
  - I.2 Dessin . . . 4
  - I.3 Écriture du code . . . 5
  - I.4 Fonction existante . . . 5
- II Méthode de Newton . . . 5
  - II.1 Principe . . . 5
  - II.2 Dessin . . . 6
  - II.3 Code . . . 6
  - II.4 Fonction existante . . . 6
- III Résolution d'un système d'équations . . . 6
  - III.1 Écriture des matrices . . . 6
  - III.2 Résolution . . . 7

2 Résolution d'équations différentielles **8**
- I Principe de la méthode d'Euler . . . 8
- II Écriture du code . . . 9
  - II.1 Premier ordre . . . 9
  - II.2 Second ordre . . . 10
  - II.3 Généralisation . . . 10
- III Exemples aux concours . . . 11
  - III.1 Une question très libre : Mines MP 2023 Physique 1 [(1)] . . . 11
  - III.2 Un code mal écrit : Centrale PC 2023 Physique 1 . . . 11

3 Incertitudes **13**
- I Estimer l'incertitude d'une grandeur qui fluctue : incertitude de type A . . . 13
  - I.1 Représentation graphique par un histogramme . . . 13
  - I.2 Incertitude-type . . . 14
  - I.3 Incertitude élargie (HP) . . . 14
  - I.4 Présentation du résultat . . . 15
- II Estimer l'incertitude d'une grandeur qui ne fluctue pas : incertitude de type B . . . 16
  - II.1 Incertitude de type B : définition . . . 16
  - II.2 De la demi-étendue à l'incertitude-type . . . 16
  - II.3 Plusieurs sources d'incertitudes pour la même grandeur (composition) . . . 17
- III Estimer l'incertitude d'une grandeur calculée . . . 17
  - III.1 Calcul . . . 17
  - III.2 Tracé de l'incertitude (TIPE) . . . 18
- IV Comparaison à une valeur attendue . . . 18
  - IV.1 Comparaison entre deux valeurs avec incertitudes . . . 18
  - IV.2 Comparaison entre une valeur expérimentale et une valeur connue sans incertitude . . . 18

V Méthode de Monte-Carlo 18
V.1 Application aux incertitudes 18
V.2 Passage en Python 19
V.3 Comparaison propagation et Monte-Carlo : Estimation de la célérité du son 19
V.4 Exemple aux écrits : dosage en chimie (d'après Centrale PC 2024) 21
VI Régression linéaire 21
VI.1 Sur Python 21
VI.2 Le R^2 est mort dans le nouveau programme 21
VI.3 Évaluation des incertitudes sur les paramètres du modèle par simulation Monte-Carlo (HP, TIPE) [2] 23

1

Résolution d'équations

I Dichotomie

I.1 Principe

Le principe de la dichotomie se base sur le théorème des valeurs intermédiaires : Si on a deux nombres a et b tels que $f(a) \times f(b) < 0$ (et que notre fonction est continue sur $[a, b]$, ce qui sera tout le temps le cas), alors il existe (au moins) un réel c tel que $f(c) = 0$.

Remarque

L'intervalle de recherche ne soit contenir qu'un seul zéro.

Exemple : Cherchons l'avancement d'une réaction du type $A + B \longrightarrow C$, de constante d'équilibre $K^\circ = \dfrac{x}{(C-x)^2}$, alors, il y a sûrement deux racines, mais seule l'une d'entre elle a un sens physique : il faut que x soit compris entre 0 et C.

Pour chercher ce nombre c, on va effectuer les opérations suivantes :

— Calculer l'abscisse m de la moitié du segment $[a, b]$ $\left(m = \dfrac{a+b}{2}\right)$

— En fonction de la valeur de $f(m)$, on ramène l'intervalle de recherche à $[a, m]$ ou $[m, b]$

— On recommence le même travail jusqu'à la précision souhaitée : $\dfrac{b-a}{2} < \varepsilon$.

I.2 Dessin

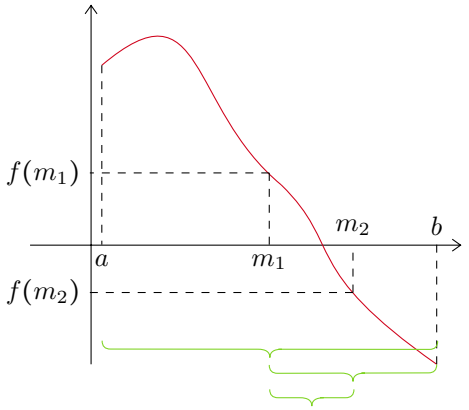


FIGURE 1.1 – Étapes successives de la méthode de dichotomie. L'intervalle d'étude (accolades vertes) devient de plus en plus petit et centré sur le zéro de la fonction.

I.3 Écriture du code

On peut assez facilement vérifier la convergence : (a_n) est une suite croissante et (b_n) est une suite décroissante.

```
def Dichotomie(f,a,b,eps):
        while (b-a) > 2*eps :
                m=(a+b)/2
                if f(a)*f(m) <0:
                        b=m
                else :
                        a=m
        return (a+b)/2
```

Algorithme 1.1 – Dichotomie

I.4 Fonction existante

La fonction **bisect** de la bibliothèque **scipy.optimize** réalise la même chose : [(1)]

`scipy.optimize.bisect(f,a,b)`

Arguments :

— `f` est une fonction continue avec $f(a)$ et $f(b)$ de signes opposés.

— `a` et `b` sont des réels

— Optionnel : `xtol` est la tolérance, c'est notre ε. S'il n'est pas précisé, il est fixé à 2×10^{-12}.

II Méthode de Newton

II.1 Principe

Si on connaît la dérivée de la fonction f, notée f' dans le cours et `fp` dans le code, alors en utilisant la tangente, on peut arriver plus vite au résultat.

La tangente a pour équation :

$$y = f'(x_0)(x - x_0) + f(x_0) \tag{1.1}$$

L'abscisse pour laquelle l'ordonnée est nulle est :

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} \tag{1.2}$$

Ainsi la relation de récurrence est :

$$\boxed{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}} \tag{1.3}$$

Remarque : On ne doit pas se retrouver avec une dérivée nulle. En pratique cela n'arrive pas si on prend x_0 pas trop loin de la racine que l'on cherche.

On peut se fixer soit :

— un nombre d'étapes N ;

— une précision : $|f(x_n)| \leq \varepsilon$

(1). Fonction abordée dans le sujet Centrale PC 2026 Physique 1

II.2 Dessin

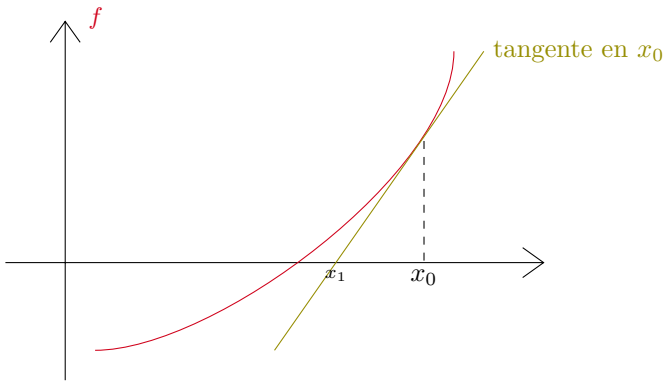


FIGURE 1.2 – Schéma de la méthode de Newton

II.3 Code

```
def Newton(f,fp,x0,n):
        xn=x0
        for i in range (n):
                xn = xn-f(xn)/fp(xn)
        return xn
```

Algorithme 1.2 – Méthode de Newton

II.4 Fonction existante

La fonction **newton** de la bibliothèque **scipy.optimize** réalise la même chose :

```
scipy.optimize.newton(f,x0)
```

Arguments :

— `f` est une fonction d'une seule variable.

— `x0` est un réel

— Optionnel : `fprime` : on peut donner ou non la dérivée.

Si f' n'est pas donné, on peut facilement le calculer avec l'approximation : $f'(x) = \dfrac{f(x+h)-f(x-h)}{2h}$.

Ce n'est pas cette méthode qui est utilisée ici, mais c'est la même idée.

— Optionnel : `fprime2` : on peut donner la dérivée seconde, cela permet d'être plus rapide notamment.

III Résolution d'un système d'équations

Cette partie sera beaucoup vue en maths (surtout avec des équations différentielles), mais nous pouvons en faire une rapide présentation.

III.1 Écriture des matrices

Si un système linéaire ressemble à quelque chose comme ceci :

$$\begin{cases} x+2y+3z=1 \\ 4x+5y+6z=2 \\ 7x+8y+9z=3 \end{cases}$$

On peut le passer sous forme matricielle en posant :

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \qquad X = \begin{pmatrix} x \\ y \\ z \end{pmatrix} \qquad B = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

On a alors $AX = B$. On a passé notre système sous forme de matrices.

III.2 Résolution

La fonction **solve** de la bibliothèque **numpy.linalg** réalise ce travail à notre place :

```
numpy.linalg.solve(a,b)
```

Arguments :

— `a` est la matrice A écrite sous la forme d'un tableau de taille (n, n).

— `b` est la matrice B écrite sous la forme d'un tableau de taille n.

Remarque : Cette méthode ne calcule pas A^{-1} pour faire $A^{-1}B$, car cela serait trop coûteux. Cette fonction se base sur une autre méthode.

2

Résolution d'équations différentielles

Considérons le problème de Cauchy suivant :

$$\begin{cases} y(t_0) = y_0 \\ y'(t) = f(y(t), t) \end{cases}$$

où y est la fonction inconnue et f une fonction de $\mathbb{R} \times I \to \mathbb{R}$ (I intervalle de $\mathbb{R}$).

Par le théorème de Cauchy-Lipschitz, on sait qu'il existe (sous certaines hypothèses) une unique solution à ce problème. En général, on ne sait pas expliciter cette unique solution, on va alors utiliser la méthode d'Euler explicite.

Remarque : On différentie explicite et implicite comme ceci :

— Euler explicite : $y_{n+1} = y_n + \Delta t \times f(y_n, t_n)$: on calcule directement y_{n+1} : explicite

— Euler implicite : $y_{n+1} = y_n + \Delta t \times f(y_{n+1}, t_{n+1})$: on résout une équation : implicite

I Principe de la méthode d'Euler

On part de t_0 et on cherche à trouver $y(t_1)$, pour cela on va utiliser la tangente en $y(t_1)$.

α) Dessin

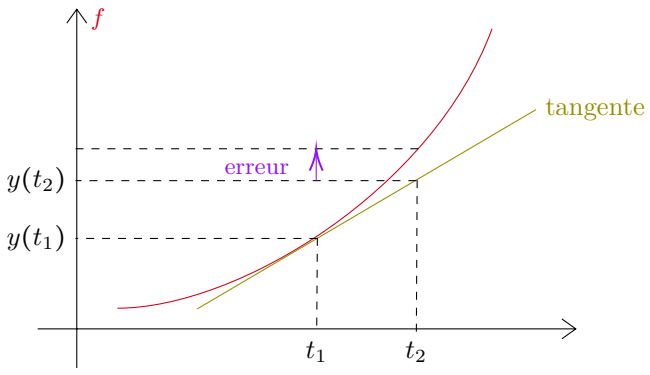


FIGURE 2.1 – Schéma de principe de la méthode d'Euler

β) Pseudo-maths

On a $y_{i+1} = y_i + \Delta y$.

On pose f la fonction qui à y et t renvoie y' et h le pas de temps choisi pour la simulation. On a

$$\Delta y = h \times f(y_i, t_i)$$

On a alors :

$$\boxed{y_{i+1} = y_i + h \times f(y_i, t_i)} \tag{2.1}$$

γ) **Application** Écrire la fonction $f(y,t)$ pour l'équation différentielle suivante : $y'^2(t) \times y^4(t) + t\sin(y(t)) = 3$
On a :

$$y'(t) = \frac{\sqrt{3 - t \times \sin(y(t))}}{y^2(t)}$$

d'où le code :

```
def f(y,t) :
        return math.sqrt(3-t*math.sin(y))/y**2
```

II Écriture du code

II.1 Premier ordre

Il ne faut pas oublier de définir f avant !

Exemple : $y'(t) = 3ty(t)$

```
def f(y,t):
        return 3*t*y
```

Remarque : Toujours mettre t même s'il n'apparaît pas dans l'expression de y'.

Le code pour résoudre une équation différentielle par la méthode d'Euler explicite s'écrit dans sa version optimisée :

```
import numpy as np

def Euler1erordre(f,y0,ti,tf,n):
        h = (tf-ti)/(n-1)
        t = np.linspace(ti,tf,n)
        y = np.ones(n)*y0
        for i in range (1,n):
                y[i] = y[i-1] + h*f(y[i-1],t[i-1])
        return t,y
```

Algorithme 2.1 – Euler Premier Ordre

Arguments :

— `f` est la fonction que l'on a définie avant
— `y0` est la condition initiale : la valeur de y à t_i
— `ti` est le temps initial, `tf` est le temps final
— `n` est le nombre de POINTS de y que l'on souhaite avoir.

Commentaires :

— On a besoin de n points, donc on a $n-1$ intervalles de temps. C'est pour cela que l'on divise par $n-1$ la différence de temps.
— Avant la boucle, on prépare nos listes, comme ça on a juste besoin de changer la valeur dans la boucle et pas en plus de créer l'emplacement dans la mémoire.
— Bien sûr, t n'est pas toujours le temps, c'est un nom de variable.
— Il est autorisé d'utiliser des listes, même si ça commence à être mal vu par les informaticiens : `t=[ti + k*h for k in range(n)]` et `y=[y0]*n`

II.2 Second ordre

Pour le second ordre, c'est à peu près la même chose :

— On définit f comme la fonction qui est égale à y''.

— On peut calculer indépendamment (sans ordre) `y[i]` et `yp[i]`.

Exemple : pour l'équation $y''(t) + 4y'(t) + 2ty(t) = 8$

```
def f(y,yp,t):
        return -4*yp - 2*t*y + 8
```

Le code pour résoudre une équation différentielle par la méthode d'Euler explicite s'écrit dans sa version optimisée :

```
def Euler2ndordre(f,yp0,y0,ti,tf,n):
        h = (tf-ti)/(n-1)
        t = np.linspace(ti,tf,n)
        y = np.ones(n)*y0
        yp = np.ones(n)*yp0
        for i in range (1,n):
                y[i] = y[i-1] + h*yp[i-1]
                yp[i] = yp[i-1] + h*f(y[i-1],yp[i-1],t[i-1])
        return t,y,yp
```

Algorithme 2.2 – Euler Second Ordre

Arguments :

— `f` est la fonction que l'on a définie avant

— `yp0` est la première condition initiale : la valeur de y' à t_i.

— `y0` est la deuxième condition initiale : la valeur de y à t_i

— `ti` est le temps initial, `tf` est le temps final

— `n` est le nombre de POINTS de y que l'on souhaite avoir.

On a alors accès au graphe de y, mais aussi de y' (utile pour des portraits de phase à tout hasard ...).

II.3 Généralisation

On peut aussi résoudre sous forme matricielle ! *Cette méthode sera vue en maths pour résoudre des équations différentielles couplées.*

Exemple : pour l'ED : $y'' + 2ty + 5y = 0$

On pose : $Y = \begin{pmatrix} y' \\ y \end{pmatrix}$ donc

$$Y' = \begin{pmatrix} y'' \\ y' \end{pmatrix} = \begin{pmatrix} -2t & -5 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} y' \\ y \end{pmatrix} = AY$$

Ainsi, on peut écrire la fonction :

```
def f(Y,t) :
        return [-2*t*Y[0] - 5*Y[1], Y[0]]
```

Cette opération permet de passer d'un système d'ordre 1 avec deux fonctions de $\mathbb{R} \to \mathbb{R}$ à 1 fonction de $\mathbb{R} \to \mathbb{R}^2$. Cette méthode fonctionne aussi pour les suites récurrentes.

III Exemples aux concours

III.1 Une question très libre : Mines MP 2023 Physique 1 [(1)]

On peut montrer que : $(\chi + \exp[W(\chi)]) \times W'(\chi) = 1$. On a $W(0) = 0$.
On souhaite appliquer le schéma d'Euler explicite avec un pas $h = 0.0001$ pour résoudre cette équation différentielle.

Question : Donner le code `Python` permettant d'obtenir une représentation graphique de $W(\chi)$ sur l'intervalle $[0; 2.5[$.

Réponse : On adapte le code du cours, en ajoutant la partie tracé de la fonction.

```
import numpy as np
import matplotlib.pyplot as plt

#Données
h = 0.0001
xmin = 0
xmax = 2.5
w0 = 0
N = int((xmax-xmin)/h)  #Nombre de points

def f(w,x):
        return 1/(x + np.exp(w))

#Création des listes
X = np.linspace(xmin,xmax,N)
W = np.ones(N)*w0

#Boucle
for i in range (1,N-1):
        W[i] = W[i-1] + h*f(W[i-1],X[i-1])

plt.plot(X,W)
plt.show()
```

Remarque : Pour le range, on va jusqu'à $N-1$ car on ne veut pas calculer la valeur en 2.5 (intervalle ouvert). Je ne sais pas si cela était pénalisé par le jury.

III.2 Un code mal écrit : Centrale PC 2023 Physique 1

L'équation différentielle obtenue est :

$$\frac{dv(t)}{dt} + av^2(t) = b \tag{2.2}$$

avec $a = 2.3 \times 10^{-3}$ et $b = 0.92$.

Le programme Python suivant est proposé pour résoudre numériquement l'équation (2.2).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 def r(f, v, t):
4         V = [v]
5         for i in range(len(t) - 1):
6                 V.append(V[i] + (t[i+1] - t[i]) * f(V[i]))
7         return V
8 def g(u):
9         return 0.92 - 2.3e-3 * u * u
```

(1). On pourra aussi aller voir le sujet Centrale PC 2026 Physique 2 (Q29 & Q30).

```
10 x = np.linspace(0, 120, 1000)
11 y = r(g, 0, x)
12 plt.plot(x, y)
13 plt.show()
```

Algorithme 2.3 – Résolution de l'équation (2.2)

Question : En explicitant les lignes concernées du programme 2.3, expliquer son principe. Quelle méthode de résolution utilise-t-il ?

Réponse : Ce code sert à résoudre l'équation différentielle (2.2), en utilisant la méthode d'Euler explicite.

- Lignes 1 et 2 : importation des packages
- Ligne 3 : `r` pour résolution, fonction permettant la résolution de l'équation différentielle avec comme argument
  - `g` la fonction dérivée (analogue à `f` dans mon cours).
  - `v` la vitesse initiale
  - `t` la liste des temps
- Ligne 4 : La liste `V` est celle qui va contenir les valeurs de la vitesse au cours du temps. On a bien `V[0]=v`.
- Ligne 5 et 6 : C'est comme dans le cours, sauf qu'ils utilisent des listes.
  Pour toutes les valeurs de temps, on calcule la vitesse de l'instant d'après grâce à la formule d'Euler.
- Lignes 8 et 9 : On définit la fonction dérivée.
- Ligne 10 : Ils appellent `x` la liste des temps ...
- Ligne 11 : On calcule toutes les valeurs de la vitesse avec une vitesse initiale nulle et pour la liste de temps établie précédemment.
- Lignes 12 et 13 : On trace le graphe.

Remarque

Ce code est mal écrit principalement car il utilise des listes, et que ce code n'est pas optimisé.
Attention à ne pas se faire avoir par ces codes qui sont écrits bizarrement !

3 Incertitudes

Définitions :

— Une **mesure** est une procédure expérimentale qui conduit à attribuer un ensemble de valeurs numériques à une grandeur, accompagné d'une unité appropriée.

— L'**incertitude** est un nombre qui quantifie la **variabilité** des résultats obtenus lors de différentes réalisations d'une même expérience dans les mêmes conditions.

Notations :

n	nombre de mesures réelles
N	nombre de tirages informatique (Monte-Carlo)
$\bar{x}$	valeur retenue
s_x	écart-type expérimental : dispersion d'une mesure
Δ	Demi-étendue de l'intervalle $[\bar{x} - \Delta; \bar{x} + \Delta]$
σ_x	écart-type d'une loi de probabilité (fourni)

I Estimer l'incertitude d'une grandeur qui fluctue : incertitude de type A

La variabilité est directement visible : on a fait n mesures $x_1, \ldots, x_n$ de la même grandeur et elles ne sont pas toutes égales.

Exemple : Mesure de $d = 10\lambda$ avec des ultrasons, répétée 10 fois :

d (cm)	7.6	7.8	8.0	7.7	7.8	7.7	7.6	7.8	7.8	7.7

I.1 Représentation graphique par un histogramme

```
plt.hist(mesures, range, bins)
```

avec :

— `mesures` : un array contenant toutes vos mesures

— `range` (optionnel) : permet d'imposer l'intervalle de l'histogramme

— `bins` (optionnel) : donne le nombre de sous-intervalle utilisé.

I.2 Incertitude-type

Lorsque l'on effectue n mesures d'une grandeur mesurée x :

— la meilleure estimation de cette grandeur est la moyenne arithmétique :

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \tag{3.1}$$

En `Python` on écrira `np.mean(mesures)`

— l'écart-type expérimental s_x quantifie la variabilité d'une unique mesure [(1)] :

$$s_x = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{3.2}$$

En `Python`, on écrira `np.std(mesures,ddof=1)`. [(2)]

— l'incertitude-type $u(\bar{x})$ quantifie la variabilité de la valeur moyenne, qui est différente de s_x (càd la variabilité d'une unique mesure) :

$$u(\bar{x}) = \frac{s_x}{\sqrt{n}} \tag{3.3}$$

Application :

```
import numpy as np
d = np.array([7.6, 7.8, 8.0, 7.7, 7.8, 7.7, 7.6, 7.8, 7.8, 7.7])  # cm
n = len(d)
dmoy = np.mean(d)
sx = np.std(d, ddof=1)
ud = sx / np.sqrt(n)
```

Algorithme 3.1 – Incertitudes de type A

Grandeur	`Python`	Question à laquelle elle répond	Valeur
d	`dmoy`	quelle valeur retenir ?	7,75 cm
s_x	`sx`	de combien varie une mesure isolée ?	0,118 cm
$u(\bar{d})$	`ud`	quelle confiance dans la moyenne ?	0,037 cm

I.3 Incertitude élargie (HP)

α) **Incertitude élargie sur une seule valeur :** En observant le graphe ci-dessous, on observe que pour une loi normale ayant pour écart-type une certaine incertitude-type $u(x)$, l'intervalle $[\bar{x} - u(x); \bar{x} + u(x)]$ ne contient que 68.3 % des valeurs.

(1). Cela a la même signification que $u(x)$ pour les incertitudes de type B. Attention, la barre est importante : $u(\bar{(x)}) \neq u(x)$.

(2). On différentie l'écart-type « mathématique » (s'écrit σ) de l'écart-type expérimental (avec un sens physique) s_x.

En mathématiques, l'écart-type σ est défini avec une division par N. En physique, si on ne réalisait qu'une seule mesure ($N = 1$), diviser par N conduirait à une incertitude-type $u(x) = 0$ . . .alors que le résultat n'est absolument pas certain ! Au contraire, diviser par $N - 1$ donne une incertitude-type non définie (0 divisé par 0), ce qui est conforme au sens physique.

L'argument optionnel `ddof=1` de la fonction `np.std` indique cette division par $N - 1$.

Sur une calculatrice aussi on différentiera σ`x` de `sx`.

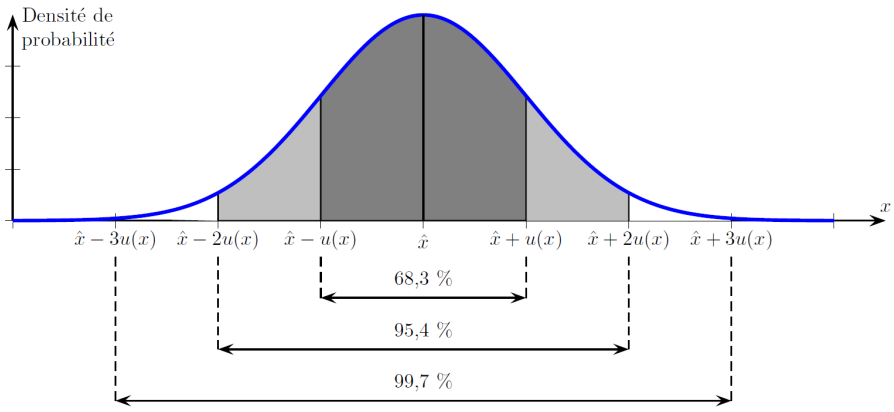


FIGURE 3.1 – Distribution normale d'une variable aléatoire

$$\text{intervalle de confiance à 95 \% : } [\bar{x}-U(x);\bar{x}+U(x)] \qquad \text{avec } U(x)=2u(x)$$

$U(x)$ est l'incertitude élargie. Cette notion s'applique aux incertitudes de type A (en prenant $U(x)=2s_x$) et aux incertitudes de type B.

β) **Incertitude élargie de la moyenne : coefficient de Student** Afin d'avoir un intervalle de confiance à 95 %, on va utiliser non plus un facteur multiplicatif de 2, mais un coefficient de Student t qui dépend du nombre de mesures effectué.

$$U(\bar{x})=t\times u(\bar{x})=t\frac{s_x}{\sqrt{N}} \tag{3.4}$$

Vous pouvez retrouver les valeurs de t sur Internet.

I.4 Présentation du résultat

L'écriture la plus courante [(3)] d'un résultat est :

$$x=\bar{x}\pm u(\bar{x})\text{ unité} \tag{3.5}$$

Méthode (sans incertitudes (la plupart des applications numériques de CPGE))

- Lorsque l'on calcule une grandeur à partir d'autres grandeurs, on cherche tout d'abord la grandeur avec le plus petit nombre de chiffres significatifs. *Rappel :* Les coefficients multiplicateurs ou les constantes des données ne comptent pas dans cette recherche.
- Écrire le résultat avec ce nombre de chiffres significatifs. (typiquement 2 ou 3)
- Afin que les chiffres significatifs ne vous emmènent pas sur des valeurs aberrantes, utilisez la touche `Ans` de votre calculatrice, ou écrivez au brouillon votre résultat avec une bonne précision (vers 4 chiffres significatifs), et utilisez ce nombre pour vos prochains calculs.

Méthode (avec incertitudes)

1. On arrondit [(4)] l'incertitude-type à 2 chiffres significatifs.
2. La valeur et l'incertitude-type doivent s'arrêter à la même décimale.
3. Attention, la moyenne et l'incertitude-type doivent avoir la même unité !

(3). En réalité, on préfère écrire l'intervalle de confiance à 95%, mais cela fait intervenir les incertitudes élargies qui sont hors-programme

(4). Il n'y a pas de règle claire sur si on arrondit au plus proche ou au supérieur.

Avec `Python`, on peut utiliser la fonction `round(x,n)` qui permet de garder n décimales du réel x.

Exemples :

— Longueur d'onde d'une raie verte du mercure : $\lambda = 546.1$ nm et $u(\lambda) = 3.17$ nm.
Cela donne : $\lambda = 546.1 \pm 3.2$ nm.

— Détermination d'une résistance électrique : $R = 4.20317$ kΩ et $u(R) = 5.236$ Ω.
Cela donne : $R = (4203.2 \pm 5.2)$ Ω.

— Distance d : $d = 7,750 \pm 0,037$ cm.

II Estimer l'incertitude d'une grandeur qui ne fluctue pas : incertitude de type B

II.1 Incertitude de type B : définition

Il existe des situations pour lesquelles reproduire l'expérience ne permet pas d'observer de variabilité :

— soit parce qu'on ne dispose que d'une observation unique ;

— soit parce que la répétition des observations conduit systématiquement à la même valeur, la variabilité intrinsèque étant masquée par la résolution limitée de l'appareil employé.



En type B, il n'y qu'une valeur lue donc $\bar{x} = x$ et $u(\bar{x}) = u(x)$.

II.2 De la demi-étendue à l'incertitude-type

Il faut estimer l'intervalle dans lequel on est (raisonnablement) certain que le résultat de la mesure se trouve, noté sous la forme $[\bar{x} - \Delta, \bar{x} + \Delta]$, dont $\bar{x}$ est appelé le centre et Δ la **demi-étendue**.

Loi	Quand ?	Incertitude-type
Uniforme (rectangulaire)	toutes les valeurs de l'intervalle sont équiprobables	$u = \dfrac{\Delta}{\sqrt{3}}$
Triangulaire	la valeur centrale est la plus probable, mais les valeurs restent confinés dans l'intervalle	$u = \dfrac{\Delta}{\sqrt{6}}$

Tableau 3.1 – Choix entre loi uniforme et triangulaire [5]

Situation	Loi
Lecture sur un cadran numérique	Uniforme
Fiole jaugée ou pipette jaugée	Triangulaire
Lecture sur graduation où le trait est sur la graduation	Triangulaire
Lecture sur graduation où le trait se situe entre deux graduations	Uniforme

Tableau 3.2 – Exemples de choix

Dans le doute, loi uniforme. C'est le choix le plus fréquent en physique, et celui qui tombe en pratique aux concours. La loi triangulaire sera utile en TIPE.

α) **Tolérance** La tolérance est analogue à la demi-étendue.

— Une résistance R de tolérance ±2 % a une incertitude-type de $\frac{0,02 \times R}{\sqrt{3}}$.

— Une solution de concentration C « précise à 1 % » peut s'interpréter de deux façons [6] :

(5). D'où viennent $\sqrt{3}$ et $\sqrt{6}$? Ce sont les écarts-types de ces lois. On pourra aussi se convaincre de $\Delta > u(x)$ par un petit schéma.
(6). Les deux se valent, même si je préfère la simulation par une loi normale, car la préparation d'une solution commerciale dépend d'un grand nombre de paramètres.

— soit on comprend précis à ±1 %, càd une loi uniforme d'incertitude-type $u(\bar{C}) = \frac{0{,}01 \times C}{\sqrt{3}}$;
— soit on comprend « d'écart-type 1 % », et dans ce cas, l'écart-type est déjà l'incertitude-type : $u(\bar{C}) = \sigma_x = 0{,}01 \times C$. (simulation par une loi normale, cf plus loin)

β) **Incertitude dûe à la méthode (un peu HP) :** La verrerie n'est pas la seule source d'incertitude, il y a aussi une incertitude dûe à la méthode.

— Pour un titrage colorimétrique, l'équivalence est repérée à le goutte près. Le volume d'une goutte étant d'environ 1/20 mL (soit 0.05 mL), l'incertitude de méthode sera 0.05 mL.
— Pour un titrage pH-métrique, l'incertitude-type sera la demi-largeur de courbe dérivée à l'équivalence.

II.3 Plusieurs sources d'incertitudes pour la même grandeur (composition)

Les incertitudes-types $(u_1, u_2, ...)$ se combinent en quadrature pour donner u l'incertitude-type totale :

$$u = \sqrt{u_1^2 + u_2^2 + \dots}$$

Exemple : Volume équivalent avec une burette ($\Delta = 0{,}03$ mL) et un repérage de l'équivalence à $\pm 0{,}05$ mL (deux lois uniformes).

$$u(V_{eq}) = \sqrt{\left(\frac{0{,}03}{\sqrt{3}}\right)^2 + \left(\frac{0{,}05}{\sqrt{3}}\right)^2} \simeq 0{,}034 \text{ mL}$$

Composition : plusieurs sources pour une même grandeur. Propagation (partie suivante) : incertitude d'une nouvelle grandeur calculée à partir d'autres.

III Estimer l'incertitude d'une grandeur calculée

III.1 Calcul

Très souvent, la grandeur d'intérêt n'est pas directement obtenue par la mesure, elle découle d'un calcul contenant les mesures.

Multiplication par une constante	$x = \lambda y \ (\lambda \in \mathbb{R})$	$u(x) = \lvert\lambda\rvert u(y)$
Somme ou différence	$x = y + z$ ou $x = y - z$	$u(x) = \sqrt{u(y)^2 + u(z)^2}$
	$x = ay + bz \ (a, b \in \mathbb{R})$	$u(x) = \sqrt{(a \cdot u(y))^2 + (b \cdot u(z))^2}$
Produit ou quotient	$x = y \times z$ ou $x = \frac{y}{z}$	$\frac{u(x)}{x} = \sqrt{\left(\frac{u(y)}{y}\right)^2 + \left(\frac{u(z)}{z}\right)^2}$
Puissances	$x = k y^p z^q \ (p, q \in \mathbb{R})$	$\frac{u(x)}{x} = \sqrt{\left(p\frac{u(y)}{y}\right)^2 + \left(q\frac{u(z)}{z}\right)^2}$

Tableau 3.3 – Formules de propagation des incertitudes

Exemple : Dosage des ions permanganate par le fer (II) : $C = \frac{2C_aV_a}{5V_0}$, on a :

$$u(C) = C \times \sqrt{\left(\frac{u(C_a)}{C_a}\right)^2 + \left(\frac{u(V_a)}{V_a}\right)^2 + \left(\frac{u(V_0)}{V_0}\right)^2}$$

Remarques :

— $u(x)/x$ est appelé l'incertitude relative.
— Il est intéressant de comparer les incertitudes relatives, afin de savoir quelle grandeur contribue le plus à l'incertitude.

III.2 Tracé de l'incertitude (TIPE)

Pour tracer un graphe avec des barres d'erreurs, on va utiliser la fonction de `matplotlib.pyplot` :

```
matplotlib.pyplot.errorbar(x, y, xerr, yerr)
```

avec :

— `x` et `y` des tableaux de données pour l'abscisse et l'ordonnée ;

— `xerr` et `yerr` soit un float pour une barre d'erreur constante, ou un tableau pour une barre d'erreur dépendante des valeurs.

IV Comparaison à une valeur attendue

IV.1 Comparaison entre deux valeurs avec incertitudes

On appelle **écart normalisé** [7] E_N entre deux valeurs x_1 et x_2 ayant une incertitude-type $u(x_1)$ et $u(x_2)$,

$$E_N = \frac{|x_2 - x_1|}{\sqrt{u(x_1)^2 + u(x_2)^2}} \tag{3.6}$$

Par convention, les deux valeurs sont **compatibles** si $E_N \leq 2$. Cela veut dire que les deux gaussiennes se recouvrent « beaucoup ».

IV.2 Comparaison entre une valeur expérimentale et une valeur connue sans incertitude

Il est assez fréquent que la valeur attendue x_a soit donnée sans incertitude (fournie par un fabricant, ou grandeur tabulée). Dans ce cas, on appelle **z-score** [8] le nombre suivant :

$$z = \frac{|x_a - x_{\text{exp}}|}{u(x_{\text{exp}})} \leq 2 \tag{3.7}$$

De même, si le z-score est inférieur à 2, on dira que les deux valeurs sont compatibles.

Pourquoi mon z-score est supérieur à 2 ? En CPGE, on sous-estime beaucoup les incertitudes, notamment en négligeant les incertitudes de méthode.

V Méthode de Monte-Carlo

Une méthode de Monte-Carlo est une méthode algorithmique utilisant le hasard pour calculer une valeur approchée.

Exemple : On tire N points (x, y) uniformément dans le carré $[0, 1]^2$. Ceux qui vérifient $x^2 + y^2 < 1$ sont dans le quart de disque. Le rapport des aires vaut $\pi/4$, donc on peut calculer π. Plus de précisions au lien suivant.

V.1 Application aux incertitudes

Idée : Chaque grandeur d'entrée est connue avec une incertitude. Au lieu d'appliquer des formules, on tire au hasard un grand nombre de valeurs possibles de chaque entrée, on calcule à chaque fois la grandeur finale, et la dispersion des résultats donne son incertitude.

(7). Le programme confond écart normalisé et score zéta (ζ), donc nous aussi. L'écart normalisé s'écrit normalement avec les incertitudes élargies et donc se compare à 1.
(8). Normalement, en français, on l'appelle plutôt score Z, mais le programme a choisi z-score.

Structure classique d'un code de type Monte-Carlo

- Importation des bibliothèques
- Définition des constantes (valeurs et leurs incertitudes)
- Tirage des N valeurs de chaque grandeur d'entrée, selon une loi centrée sur sa valeur et dont la dispersion traduit $u(\bar{x})$.
- Calcul de la grandeur voulue par opérations sur les tableaux.
- Calcul de la moyenne et de l'écart-type avec `numpy.mean(...)` et `numpy.std(... , ddof=1)`.

Loi	Normale	Uniforme	Triangulaire
Utilisation	Valeur pas « confinée » entre deux valeurs ; ou si la grandeur dépend d'un nombre important de paramètres	Toutes les valeurs comprises dans l'intervalle ont la même probabilité	Valeurs dans un intervalle, mais avec la valeur centrale la plus probable
Incertitude-type (pour formule propagation)	$u = \sigma$ (si fourni, type B)	$u = \dfrac{\Delta}{\sqrt{3}}$	$u = \dfrac{\Delta}{\sqrt{6}}$
Paramètre à donner à numpy	$u(\bar{x})$, càd σ si type B, ou $s_x/\sqrt{n}$ si type A	Δ	Δ

Tableau 3.4 – Choix de la loi de probabilité

A l'écrit, la loi imposée n'est pas toujours la plus réaliste : **suivre l'énoncé**.

V.2 Passage en Python

α) Lois usuelles en type B :

Normale (centre x ; écart-type u)	`np.random.normal(moyenne, écart-type, N)`
	`np.random.normal(x, u, N)`
Uniforme (centre x, demi-étendue Δ)	`np.random.uniform(min, max, N)`
	`np.random.uniform(x - Delta, x + Delta, N)`
Triangulaire (centre x, demi-étendue Δ)	`np.random.triangular(min, pic, max, N)`
	`np.random.triangular(x - Delta, x, x + Delta, N)`

Tableau 3.5 – Notation `Python` des lois de probabilité

β) Moyenne de n mesures (type A) :

On ne met pas s_x seul car on met $u(\bar{x})$:

```
np.random.normal(x, sx/np.sqrt(n), N)
```

γ) Plusieurs sources d'incertitudes :

Si on a plusieurs sources d'incertitudes pour une même grandeur, alors on a juste à sommer les fonctions `np.random`, comme dans l'exemple du V.4.

V.3 Comparaison propagation et Monte-Carlo : Estimation de la célérité du son

Un émetteur émet en continu des ultrasons à la fréquence $f = 44.1$ kHz, reçus par deux récepteurs séparés d'une distance d. Lorsque la distance d est un multiple de la longueur d'onde λ, alors les signaux reçus par les deux récepteurs observés sur l'oscilloscope sont en phase. Partant des deux récepteurs côte à côte, on en recule un jusqu'à compter dix passages en phase : ils sont alors séparés de $d = 10\lambda$. D'après la relation de dispersion,

$$c = \lambda f = \frac{df}{10}$$

En reproduisant dix fois l'expérience, on mesure successivement :

d (cm)	7.6	7.8	8.0	7.7	7.8	7.7	7.6	7.8	7.8	7.7

La notice de l'émetteur d'ultrasons indique que la fréquence est comprise avec certitude dans l'intervalle [44.0 kHz, 44.2 kHz].

α) **Méthode de Monte-Carlo :** Compte tenu de la méthode de mesure, la distance d sera tirée aléatoirement suivant une loi gaussienne. [(9)] Au contraire, la fréquence f sera tirée aléatoirement selon une loi uniforme.

```
import numpy as np

# Valeurs expérimentales
d_exp = np.array([7.6,7.8,8.0,7.7,7.8,7.7,7.6,7.8,7.8,7.7])*1e-2
n = len(d)

# Type A : on simule la MOYENNE de d
d_moy = np.mean(d_exp)  # Moyenne
u_d = np.std(d_exp, ddof=1)/np.sqrt(n)  # Incertitude-type

# Type B : intervalle certain pour f
f_min, f_max = 44.0e3, 44.2e3

# Tirages
N = 100000
d = np.random.normal(d_moy, u_d, N)
f = np.random.uniform(f_min, f_max, N)
c = d*f/10

# Résultats
c_moy = np.mean(c)
u_c = np.std(c, ddof=1) # Pas de division par sqrt(N)

# Affichage
print("célérité moyenne :",c_moy,"incertitude :", u_c)
```

Algorithme 3.2 – Monte-Carlo appliqué à la physique

On obtient : $c_{moy} = 341.7725121297$ m/s et $u(c) = 1.705168607001$ m/s, soit $c = 341,8 \pm 1,7$ m·s^{-1}.

β) **Formule de propagation :**

— Pour d, on a la moyenne qui vaut : $d = 7,75$ cm, l'écart-type expérimental égal à $s_x(d) = 0,118$ cm et l'incertitude-type égale à $u(d) = 0,037$ cm.

— Pour f, la valeur moyenne est $f = 44.1$ kHz, et l'incertitude-type est $u(f) = \dfrac{0.1}{\sqrt{3}} = 0,058$ kHz

— Par la formule, puis celle de propagation, on a :

$$c = \frac{df}{10} = 341,8 \text{ m}\cdot\text{s}^{-1} \quad \& \quad u(c) = c \times \sqrt{\left(\frac{u(d)}{d}\right)^2 + \left(\frac{u(f)}{f}\right)^2} = 1,7 \text{ m}\cdot\text{s}^{-1}$$

soit $c = 341,8 \pm 1,7$ m·s^{-1}.

On obtient la même chose, et c'est normal !

(9). (HP) En réalité, lorsque l'estimation d'une incertitude-type d'une grandeur provient de n observations répétées, la loi de probabilité n'est pas une gaussienne mais une loi de Student à $n-1$ degrés de liberté.

V.4 Exemple aux écrits : dosage en chimie (d'après Centrale PC 2024)

On dose l'ibuprofène, dont la masse est calculée par : $m = C_2(V_{eq2} - V_{eq1})M$, avec $C_2 = 0,100$ mol/L, $V_{eq1} = 9,0$ mL, $V_{eq2} = 20,5$ mL et $M = 206,27$ g/mol.

Par une analyse de sa procédure, l'expérimentateur est amené à considérer les données suivantes :

— précision des solutions fournies par les techniciens : 1% (loi normale [(10)]) ;

— indication sur la burette graduée : ±0.03 mL (loi uniforme) ;

— demi-étendue associée aux volumes équivalents estimée à ±0.05 mL (loi uniforme) ;

On demande la simulation Monte-Carlo donnant l'incertitude-type sur m.

```
import numpy.random as rd

# Données
C2, Veq1, Veq2, M = 0.100, 9.0e-3, 20.5e-3, 206.27

# Ecart-type / incertitude-type pour loi normale
u_C2 = C2/100
# Demi-étendues Delta pour loi uniforme
Delta_Vb = 0.03e-3
Delta_Veq = 0.05e-3

# Tirages
N = 10000
C2_sim = rd.normal(C2, u_C2, N)
Veq1_sim = Veq1 + rd.uniform(-Delta_Vb, Delta_Vb, N) + rd.uniform(-Delta_Veq,
    Delta_Veq, N)
        # Somme des sources d'incertitudes
Veq2_sim = Veq2 + Delta_Vb*rd.uniform(-1, 1, N) + Delta_Veq*rd.uniform(-1,1,N)
        # Écriture donnant la meme chose qu'au-dessus

# Masse d'ibuprofène
m_sim = C2_sim*(Veq2_sim-Veq1_sim)*M
u_m = np.std(m_sim, ddof=1)
```

Algorithme 3.3 – Code Monte-Carlo Centrale PC 2024

VI Régression linéaire

VI.1 Sur Python

`numpy.polyfit(x,y,deg)`

avec `x` les valeurs en abscisse, `y` les valeurs en ordonnée et `deg` le degré du polynôme avec lequel vous voulez fitter.

Cette fonction renvoie un tableau avec les coefficients du polynôme, en commençant par l'ordre le plus haut.

Exemple : Pour une droite $y = ax + b$, on écrit `a, b = np.polyfit (x, y, 1)`.

VI.2 Le R^2 est mort dans le nouveau programme

Le coefficient de corrélation linéaire R^2 que vous aviez l'habitude de manipuler est bien, mais peut parfois donner des valeurs absurdes [(11)]. De plus, il ne prend pas du tout en compte les incertitudes.

Comment valider une régression linéaire ?

(10). Dans le sujet originel, ils ont pris une loi uniforme mais je voulais changer.
(11). On pourra notamment s'intéresser au quartet d'Anscombe (lien)

VI.2.a Validation visuelle

Méthode

— *Si on a pas d'incertitudes :* En première approche, le modèle sera validé si, à l'oeil, les points de mesure sont bien alignés et que la droite passe au plus près des points.

Rédaction aux écrits (Rapport Centrale PC 2023 Chimie)
Exemple : La régression linéaire de ... en fonction de ... donne une droite d'équation $y = \ldots x + \ldots$. On observe que les points sont bien alignés avec la régression linéaire (et $R^2 = \ldots$), on en conclut que ...

— *Si on a les incertitudes :* La meilleure régression linéaire est celle dont TOUTES les barres d'incertitudes chevauchent la droite de régression linéaire, et que les points soient répartis aléatoirement autour de la droite de régression.

α) **Exemple : Centrale PC 2024 Chimie** Dans l'optique de proposer un modèle cinétique qui permette de décrire convenablement les résultats expérimentaux, on trace les nuages de points suivants et les barres d'incertitude associées :

A en fonction du temps (figure 3.2) ; $\ln A$ en fonction du temps (figure 3.4) ; $\frac{1}{A}$ en fonction du temps (figure 3.3)

Les droites de régression et leur équation ont été également ajoutées à l'aide du logiciel `Python`.

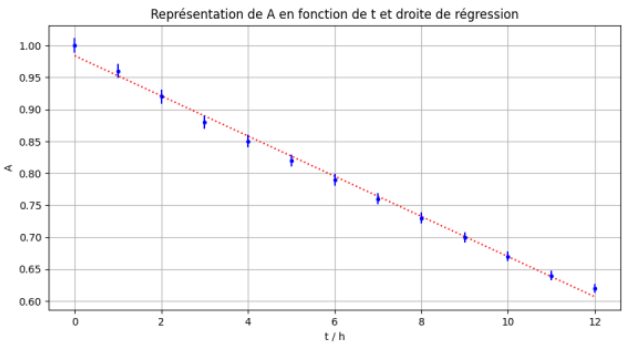


FIGURE 3.2 – Tracé de A en fonction de t

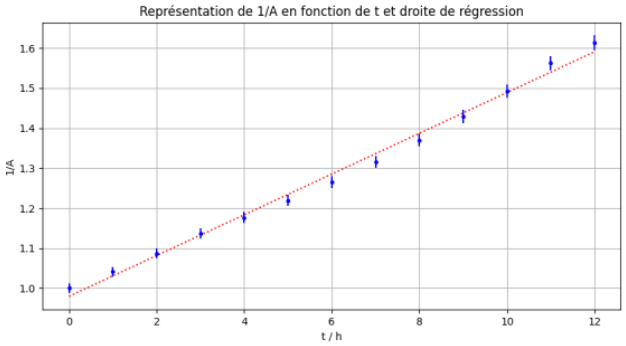


FIGURE 3.3 – Tracé de $\frac{1}{A}$ en fonction de t

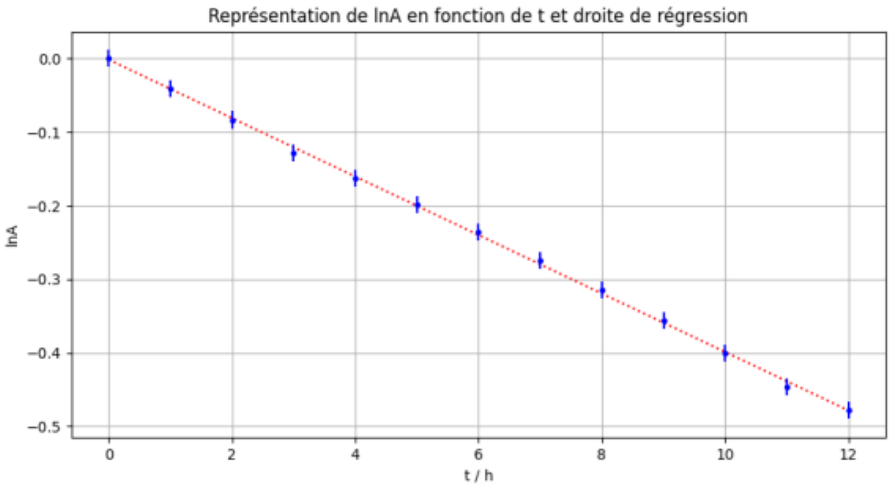


FIGURE 3.4 – Tracé de $\ln A$ en fonction de t

Question : *(implicitement posée)* Donner l'ordre de la réaction.

Réponse : On voit que la seule courbe dont les barres d'incertitudes chevauchent tout le temps la régression linéaire est la figure 3.4. Ainsi, la réaction est d'ordre 1.

!! On voit ici les limites du coefficient de corrélation linéaire R^2. Les trois régression linéaires sont plutôt alignées avec les points, donc les R^2 doivent être proches de 0.99.

Si vous aviez supposé un ordre 2 et eu un R^2 à 0.98, vous auriez conclu que c'était bien un ordre 2, alors que non. ET vous n'auriez pas eu le réflexe de regarder les autres ordres, vu que pour 2 « ça marche bien ». Voilà comment on peut montrer des choses fausses.

VI.2.b Tracé des résidus

Parfois les barres d'incertitudes sont trop petites pour que l'on voit bien si les barres et la régression linéaire se chevauchent bien. Pour cela on va tracer les « résidus », c'est-à-dire : $y_i - (a \cdot x_i + b)$ (= valeur - valeur trouvée par la régression). Les incertitudes restent les mêmes.

VI.2.c Tracé des z-scores / écarts normalisés [12]

Comme on l'a vu avant, on peut tracer les z-score de chaque point : $z_i = \dfrac{y_i - (a \cdot x_i + b)}{u(y_i)}$

Il suffit alors de vérifier que tous les z-scores (z_i) soient compris entre -2 et 2.

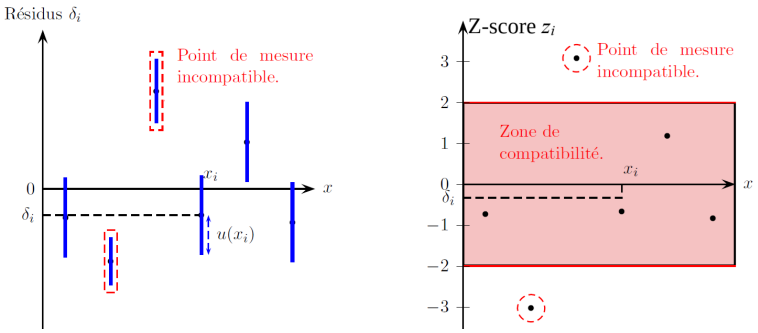


FIGURE 3.5 – Tracé des résidus et des z-scores

VI.3 Évaluation des incertitudes sur les paramètres du modèle par simulation Monte-Carlo (HP, TIPE) [13]

Méthode

On dispose de n points de coordonnées (x_i, y_i), d'incertitudes $(u(x_i), u(y_i))$.

1. On fixe un nombre N de simulations à réaliser (avec $N \gg 1$).
2. A chaque itération (pour k allant de 1 à N) :
   - Pour chaque point i, on tire aléatoirement (selon la loi choisie) un nouveau point $(x_{i,k}, y_{i,k})$.
   - Faire cela pour les n points.
   - Faire une régression linéaire. On obtient alors les coefficients a_k et b_k.
3. Extraire les valeurs moyennes et incertitudes-types de a et b avec `np.mean()` et `np.std()`.

(12). Cf Centrale PC 2025 Chimie ; le sujet appelle cela un écart normalisé alors que c'est un z-score ...
(13). On pourra aller voir le code de la question 15 du sujet Centrale PSI 2026 Physique-Chimie 1.